Spectral Clustering: Intuition and Implementation

Ming Xiao
Dept. of Computer Science
University of California, Davis
Davis, CA 95618
Email: minxiao@ucdavis.edu

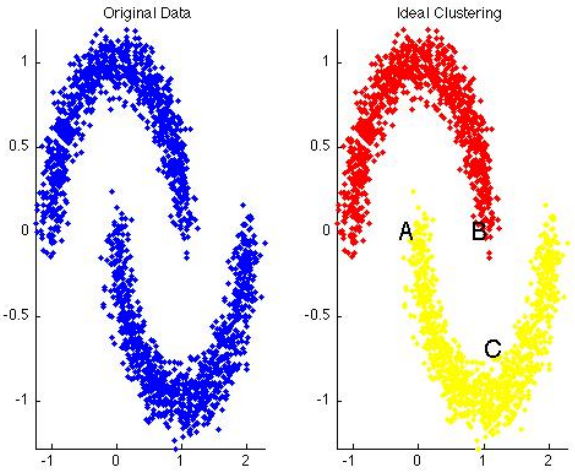

Fig. 1. On the left we have our original dataset. On the right we have the ideal paritition.

***Abstract*—Clustering is an important task that pertains to many areas. Spectral clustering is one clustering method. We will present some intuition on what it is, then go into a high level overview of the algorithm with experimental results, along with psuedocode and implementation details. This is meant to be a lighthearted overview.**

I. Motivation

There are many tasks out there that require some sort of categorization, paritition, or grouping of data. Businesses could partition their customers into different groups to provide more customized services. Social network could group people together and suggest new friends.

Those task call for some sort of clustering, or grouping of things. Spectral clustering is one such method.

II. Intuition

Spectral clustering is best explained through pictures. Take the left plot of Figure 1, if you stopped someone on the street and said, "Hey! I'm a CS PhD student, could you split these points up into two groups?", they would probably produce the groups shown on the right plot.

Though that's easy enough for us humans to split, there are some non-intuitive things going on. We typically group things that are close together. The closer they are, the more related they must be.

But notice points A, B and C on the right plot. A is definitely closer to B than C, but A and C are grouped together! Of course there are some reason for this. If you asked the same person why they split it that way, even though A and B are closer, they would say something like "Look, A and B are part of the same 'chain', duh! Are you really a PhD student?"

And this is the visual intutition behind spectral clustering we want to group things together that are some how 'linked'. How do we determine which things are 'linked'? It's a good thing you asked, otherwise this would be a very short paper...

III. Similarity Measure

Given our dataset $X = x_1, ..., x_n$, we want compute the similarity measure between each pair of points. We a graph $Gr = (V, Edge)$, where the vertex $v_i \in V$ are our datap points x_i. We then construct our edge set $Edge$, through some sort of similarity measure between vertices. There are many different similarity measures, such as:

- **Euclidean Distance**: The Euclidean distance between two points p, q is $\sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$. [3] This is the most intuitive way to deducing similarity.
- **k-nearest neighbor graphs**: Here the goal is to connect vertex v_i with vertex v_j if v_j is among the k-nearest neighbors of v_i . However, this denition leads to a directed graph, as the neighborhood relationship is not symmetric. There are two ways of making this graph undirected.
  The first way is to simply ignore the directions of the edges, that is we connect v_i and v_j with an undirected edge if v_i is among the k-nearest neighbors of v_j or if v_j is among the k-nearest neighbors of v_i . The resulting graph is what is usually called the k-nearest neighbor graph. Think of the arkward situation where you consider someone your neighbor, but they don't consider you to be their neighbor. So two vertices are neighbors is *any* one of them says they are.
  The second choice is to connect vertices v_i and v_j if both v_i is among the k-nearest neighbors of v_j and v_j is among the k-nearest neighbors of v_i . The resulting graph is called the mutual k-nearest neighbor graph. Two vertices are neighbors if *both* of them say they are. [7]

For our purposes we are going to use the **gaussian distance**. The gaussian distance between two points $g(x_i, x_j) =$

$e^{\frac{(x_i-x_j)^2}{2\sigma^2}}$. [1] Where σ represents how 'wide' of a neighborhood to consider. This value is typically 1.

Why did we choose this metric? This metric says that if two points are close together, then they are *really* related. Whereas other metric might only say they are only somewhat related. The gaussian distance follows a probability distribution so we know $0 \leq g(x_i, x_j) \leq 1$. The closer that value is to 1, the more related they are.

Let us create a matrix G, where $G_{ij} = g(x_i, x_j)$. Let us picture what G looks like.

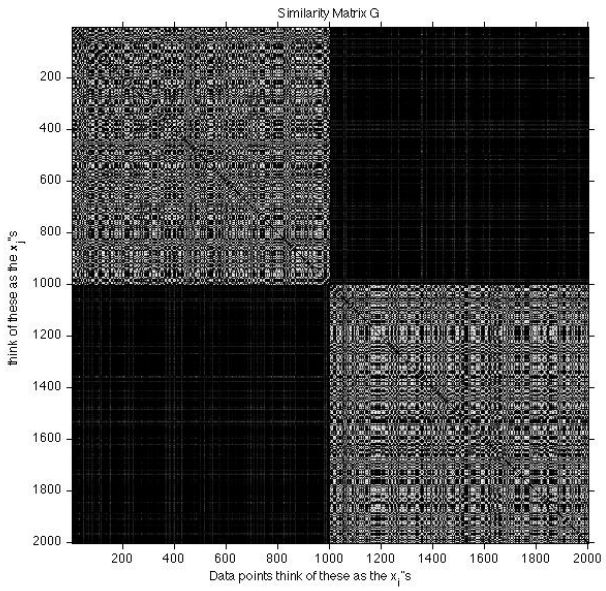

Fig. 2. Gaussian distance between each pair of points

Figure 2 shows our similarity matrix. G, isn't that nice? In our data set there are 2000 points. So G is a 2000x2000 matrix. Each point in Figure 2 represent the similarity between two points. White means that the value is closer to 1, meaning they are similar. And black means the value is closer to 0, meaning they are dissimlar.

We can see two square 'white-ish' blocks. One block is in the upper left hand corner consisting of points 0-1000. Meaning the points in those blocks are similar. Of course this example was set up so that we can see the separation clearly.

We also need to calculate the degree matrix D. D is a diagonal matrix and is defined to be: $D_{ii} = \sum_{j=1}^{n} G_{ij}$.[8]

In other words the i^{th} diagonal element of D is the sum of the i^{th} row of G. We will be using D and G to calculate something called the Laplacian graph.

IV. Laplacian Graph

The Laplacian Graph $L = D - G$. In plain words the Laplacian is the difference between the degree matrix and the similarity matrix.

Why do we care about the Laplacian? As it turns out, the Laplacian has the property that it is positive-semidefinite.

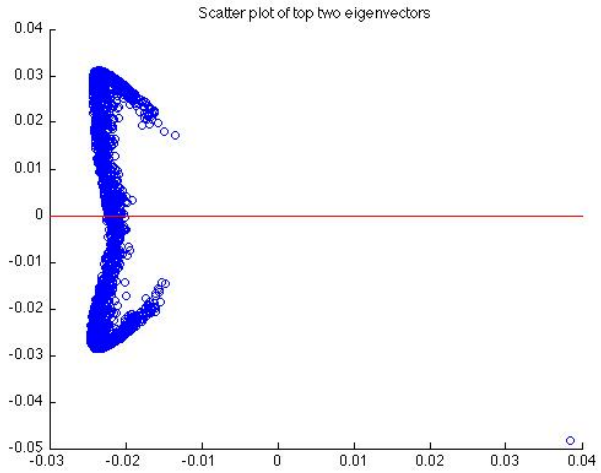

Fig. 3. Scatter plot of top two eigenvalues

That means we can decompose it into its eigenvectors and eigenvalues.[7]

Back at the beginning of this report, we talked about the inutition that some points just fell into the same 'chain'. Well these eigenvectors are the representation of those 'chains'.

Our Laplacian L has 2000 eigenvectors ,but we don't need that many. Refering back to Figure 1, we see that we have two groups or 'chain' of points, so we just want to extract the dominant two eigenvectors!

Another way to think about is is that the Laplacian tells us how 'connected' our original dataset was. Extracting two eigenvectors means give us two 'connected' components. We see that there is a direct connection between the number of eigenvectors we need to extract and the number of clusters we want.

Before we move forward we need to compute the *normalized* Laplacian $\hat{L} = D^{-1/2}LD^{-1/2}$.[7] We normalize so that each entry is scaled down to an appropriate range. The next step is to extract the eigenvectors of $\hat{L}$.

V. Extracting Eigenvectors

Let E be the matrix consisting of the largest two eigenvectors of $\hat{L}$. In our case E is a 2000x2 matrix. Figure 3 is a scatter plot of each row of matrix E. We see that we can split those points into approximately two groups at the red line.

This is significant because the points *below* the red line will all map to one group, and the points *above* the red line will map to another group. This is exactly what we wanted, to split out dataset into two groups where each points in each group are connected by a 'chain'.

We will now use the K-Means clustering algorithm [5] to cluster the points in Figure 3 into two separate groups. That's right the spectral clustering algorithm, uses another clustering algorithm!

Well, its fairer to say that in spectral clustering we are essentially remapping our data. Instead of clustering on our

original dataset. We cluster on the top K eigenvectors of that dataset.

VI. K-Means Clustering

A. Overview

K-means clustering consist of initialization and then repetition of two steps: assignment and update.

We initialize k random points to be the cluster centroids, k being the number of clusters we want.

In the assignment step, each data point is assigned to its closest centroid.

In the update step, we move the centroid to the "center" of those points assigned to it.

We repeat the assignment and update step until the centroid stops moving.

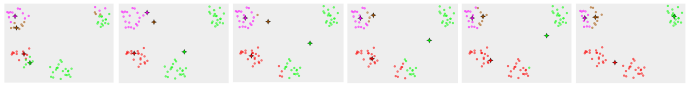

Fig. 4. Illustation of K-means algorithm. The leftmost plot is initialization and assignment. Next we update, by moving the centroids. Then we assignment, then update again.

As with most things, a illustration is much easier to understand. [6] Figure 4 shows the steps of K-means.

A more detailed overview is included in the Appendix.

B. Result

Figure 5 shows the result of K-means clustering on the points of the eigenvectors. We can see that the clusters are split at roughly the line y=0, which is what we predicted.

Remember that, in our scenario, there were 2000 data points in our original dataset. How many points are in the scatter plot of the rows of E (See Figure 3)? You guessed it, 2000!

This shouldn't be surprising. Let's backtrack the process. Our normalized Laplacian $\hat{L}$ was 2000x2000. And we took the top two eigenvectors of $\hat{L}$ and put that into a matrix$E$, which is 2000x2.

Each point (row vector) in E corresponds to a point in our original dataset. Having clustered E we can use the cluster assignment of E on our original dataset, this will yield the result in Figure 6.

VII. Remarks and Limitations

We see that our spectral clustering algorithm did not cluster our dataset into the ideal clusters we were looking for. The inner tip of each 'moon' has been absorbed by the other cluster.

This limitation is caused by the use of the K-means clustering algorithm.

Figure 7 illustrates this subtle point. In each of the "Ideal Clustering" plots the red cluster has 100 points, while the yellow cluster has 300 points.

In the first row of plots we see that the clustering does what we wanted, two separate clusters of different sizes. However when the clusters get close enough together, we have an issue.

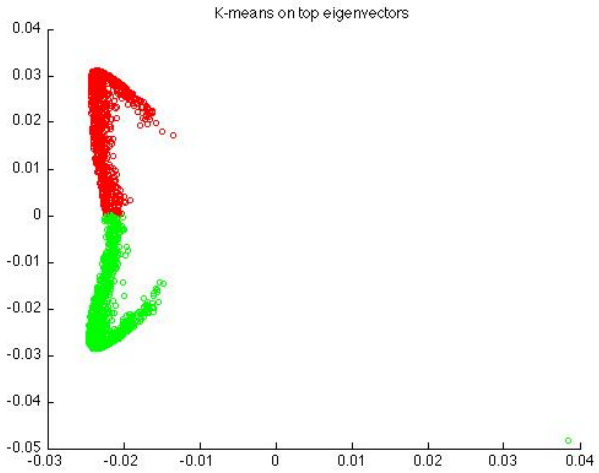

Fig. 5. Points of the eigenvectors clustered using K-means

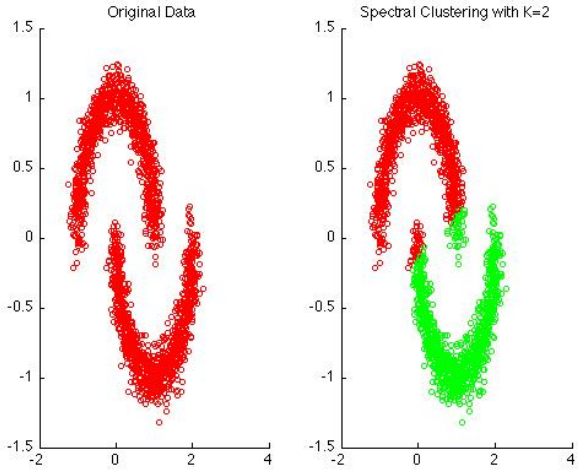

Fig. 6. Spectral clustering results

When the clusters are close, the smaller cluster 'eats' into the larger cluster. Refering to our plot of the eigenvalues in Figure 3, this is clearly an issue since those clusters are share a direct border.

Another limitation is that we may *not* get the same results using spectral clustering every time! This limitation is also due to the K-means clustering algorithm. In K-means, we choose random points as our initial centroids.

Since the initial points are randomly chosen, we have no guarantee on the final centroid locations.

VIII. Experiment

A. Introduction

We will see how spectral clustering performs on several oddly shaped datasets, shown in Figure 8. Set 1 consist of 399 vectors, and 6 clusters. Set 2 consist of 300 vectors and 3 clusters. Set 3 consist of 312 vectors and 3 clusters. These sets were taken from [2].

We expect spectral clustering to perform well on Set 2 and especially Set 3. It seems that Set 3 is why spectral clustering was created! Set 1 on the other hand, will cause difficulties.

There are several different patterns of clusters in Set 1. We see the inner/outer blue clusters in the lower right, the scatter/condensed cluster on the right, and the "normal" adjacent clusters on the upper left hand corner. Spectral clustering was not meant to detect all these patterns, so I expect it to perform poorly.

B. Result and Analysis

As discussed in the Limitations section, each run of spectral clustering may produce different results. Figure 9 are the best results that was produced.

As expected spectral clustering clustered Set 3 perfectly. When we look at the clustering of the rows of the top eigenvectors of Set 3, Figure 10 we see why that is. The points of the eigenvectors are clustered perfectly. Keep in mind this was the best result produced. There were several occasions in which substantially inaccurate results were produced.

Spectral Clustering did not perform optimally on Set 2. We see that it did recognize the two inner clusters. However it was unable to recognize the outer ring. An intersting note is that there are a few red points scattered in *between* the blue and green clusters!

Figure 11 illustrates one of the limitations of spectral clustering. We can clearly see that there are three distinct clusters, however K-means was not able to correctly recognize them. Probably due to a bad initial guess.

This leads us to belive that with enough runs, spectral clustering will arrive at the optimal cluster configuration of Set 2.

As expected Set 1 performed the worst. Although it did recognize the outer ring in the lower left hand corner, it was not able to recognize anything else. Unfortunately the points of the eigenvectors of Set 1 are in 6-dimensional space so we cannot analyze it visually.

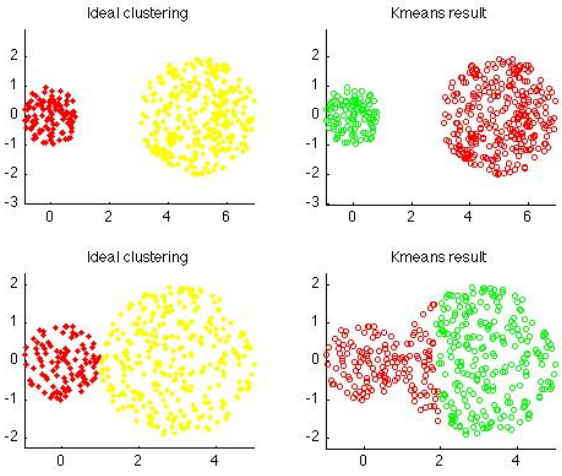

Fig. 7. The limitations of k-means clustering

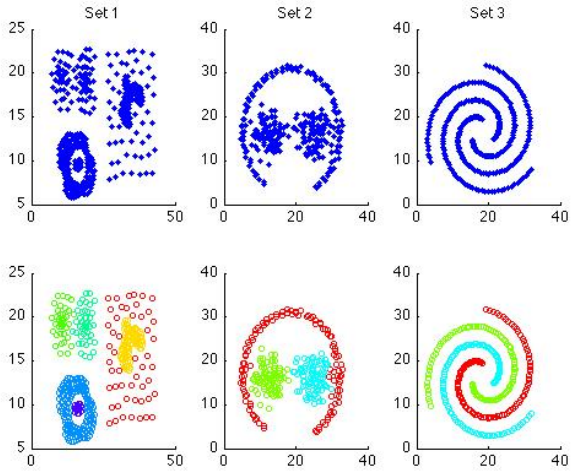

Fig. 8. The first row consist of data sets used in the experiment. The corresponding plots in the second row are the given correct clusters.

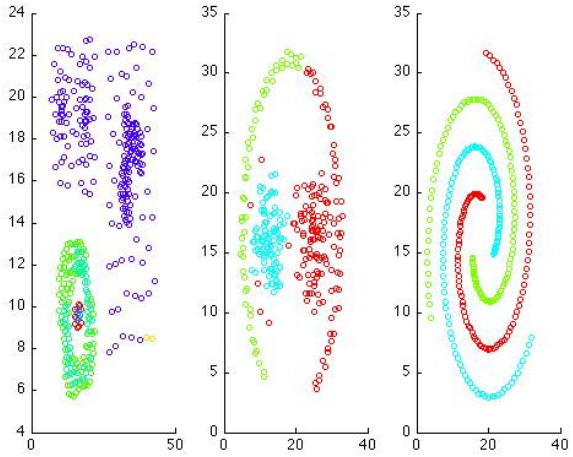

Fig. 9. Results from spectral clustering

IX. Implementation Details

MATLAB has built in function so do much of the heavy lifting of this algorithm. Most notably you can use *eig* to calculate the eigenvectors and *kmeans* to get the cluster assignments.

To the best of my knowledge the similarity matrix is one that you have to compute by yourself. As the size of the input grows, that bottle neck is creating the similarity matrix and calculating the eigenvectors.

The naive creation of the similarity matrix will have two loop, nested. In the form of:

for i =1:n

  for j = 1:n

    A(i,j) = similarity of x_i, x_j

You can speed this up using a vectorized implementation.

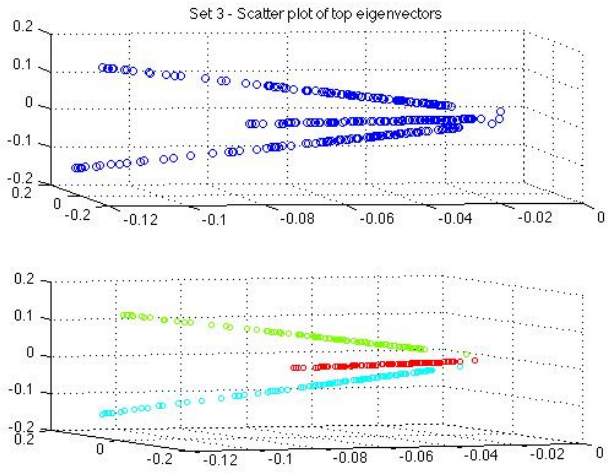

Fig. 10. Set 3 - Clustering of points of eigenvectors

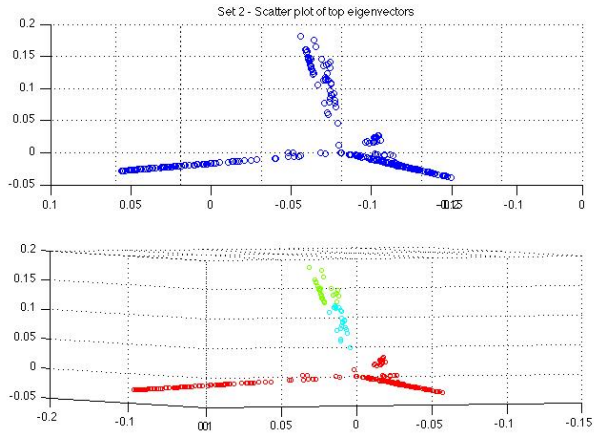

Fig. 11. Set 2 - Clustering of points of eigenvectors

X. Conclusion

Spectral clustering provides a intuitive way to clustering objects off oddly shaped 'chains'. It is built on top of K-means clustering, in that we remap our data into a new dataset partitioned by K-means.

Once the theory is understood it is easy to implement especially in a higher order language such as MATLAB, where much of the heavy lifting is already provided.

An issue worth further investigation is that can we do some sort of nested spectral clustering? If we look at Figure 11, it is clear that if we did spectral clustering on point of the eigenvectors, when we would have achieved the optimal clusters.

References

[1] George Casella. *Statistical Inference*. Thomson Learning, 2002.

[2] Hong Chang and Dit-Yan Yeung. Robust path-based spectral clustering. *Pattern Recognition*, 41(1):191 – 203, 2008.

[3] Elena Deza. *Encyclopedia of distances*. Springer Verlag, Dordrecht New York, 2009.

[4] A. K. Jain, M. N. Murty, and P. J. Flynn. Data clustering: a review. *ACM Comput. Surv.*, 31(3):264–323, September 1999.

[5] David Mackay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2004.

[6] E.M. Mirkes. K-means and k-medoids applet, 2011.

[7] Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17:395–416, 2007. 10.1007/s11222-007-9033-z.

[8] Eric W. Weisstein. "laplacian matrix." from mathworld–a wolfram web resource, June 2012.

XI. Appendix

A. K-means Algorithm

Algorithm 1: K-Means Algorithm

Input: $E = \{e_1, e_2, \ldots, e_n\}$ (set of entities to be clustered)
k (number of clusters)
$MaxIters$ (limit of iterations)
Output: $C = \{c_1, c_2, \ldots, c_k\}$ (set of cluster centroids)
$L = \{l(e) \mid e = 1, 2, \ldots, n\}$ (set of cluster labels of E)

foreach $c_i \in C$ **do**
  $c_i \leftarrow e_j \in E$ (e.g. random selection)
end
foreach $e_i \in E$ **do**
  $l(e_i) \leftarrow argminDistance(e_i, c_j) j \in \{1 \ldots k\}$
end

$changed \leftarrow false$;
$iter \leftarrow 0$;
repeat
  foreach $c_i \in C$ **do**
    $UpdateCluster(c_i)$;
  end
  foreach $e_i \in E$ **do**
    $minDist \leftarrow argminDistance(e_i, c_j)\ j \in \{1 \ldots k\}$;
    if $minDist \neq l(e_i)$ **then**
      $l(e_i) \leftarrow minDist$;
      $changed \leftarrow true$;
    end
  end
  $iter++$;
until $changed = true$ *and* $iter \leq MaxIters$;

Fig. 12. The K-means Algorithm [4]

B. PsuedoCode

X = dataset n-by-m
A = Similarity(X) n-by-n
calculate D
L = D-A
$\hat{L} = D^{-1/2} L D^{-1/2}$
E = top k eigenvectors of $\hat{L}$
cluster assignments = K-means clustering on E

C. MATLAB Implementation

```
% X is our data n-by-m
K=2
m = size(X,1);
A = zeros(m,m);
sigma = 1;

for i = 1:m
for j =i:m
if i==j
A(i,j) = 0;
        else
A(i,j) = exp(-sum((X(:,i)-X(:,j)).^2)/(2*sigma^2));
        end
    end
end

%% Compute the degree matrix
D = size(A);
for i=1:size(A,1)
    D(i,i) = sum(A(i,:));
end

L = D^(-1/2)*A*D^(-1/2);

[eigVector eigValue] = eig(L);

nVector = eigVector(:[1:k]);

[idx centroids] = kmeans(nVector,2);

% idx represent the cluster assignments
```